DMV PRACTICALS 1 TO 7

Algorithms and Flowcharts

Data Mining and Visualization (DMV)

A revision booklet containing step-by-step algorithms and vertical flowcharts for the seven DMV practical topics discussed.

Practical 1	Data Loading, Storage and File Formats
Practical 2	Working with OpenWeatherMap API
Practical 3	Data Cleaning and Preparation — Customer Churn
Practical 4	Data Wrangling on Real Estate Market
Practical 5	Data Visualization — Air Quality
Practical 6	Data Aggregation — Retail Sales
Practical 7	Time Series Analysis — Stock Market

Note: The algorithms and flowcharts are based on the practical topics and code shared in this conversation. Practical 1 and 2 are summarized from the previously discussed topics; adjust any step if your official lab manual specifies a different dataset or sequence.

Practical 1: Data Loading, Storage and File Formats

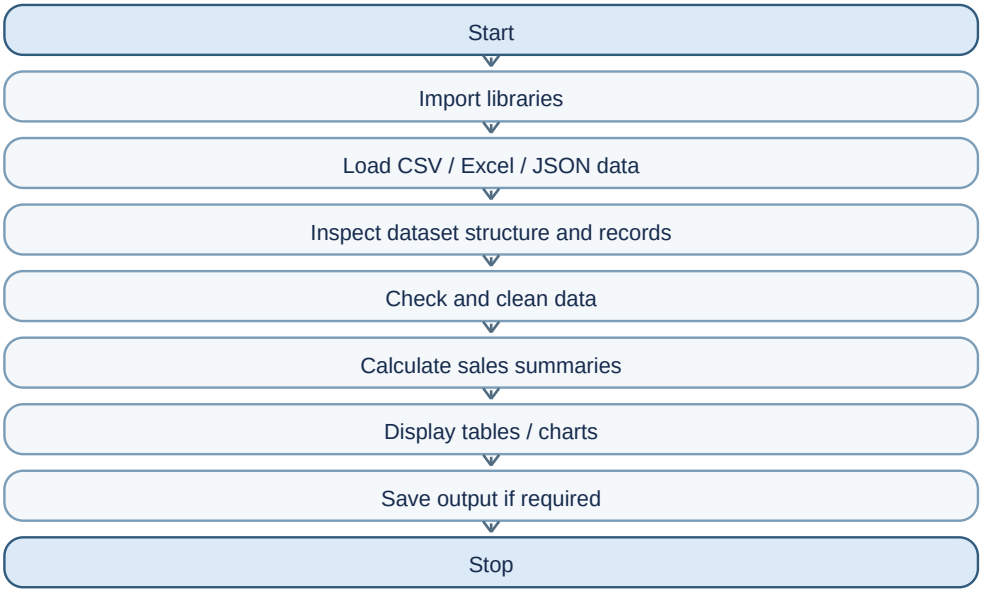
Problem statement

Load and inspect data stored in common file formats such as CSV, Excel, and JSON, then perform a simple sales-data analysis and present the results.

Algorithm

1. Start the program.
2. Import the required libraries, mainly Pandas and visualization libraries if needed.
3. Load the sales dataset from the provided CSV, Excel, or JSON file.
4. Inspect the data using head(), tail(), info(), shape, columns, and describe().
5. Check for missing values and duplicate records.
6. Clean or prepare the data as required for analysis.
7. Calculate relevant summaries, such as sales totals or category-wise values.
8. Display the results in tables and/or charts.
9. Save the processed data or output file if required.
10. Stop.

Flowchart



Practical note: Use the exact file formats and sales columns supplied in your practical dataset.

Practical 2: Working with OpenWeatherMap API

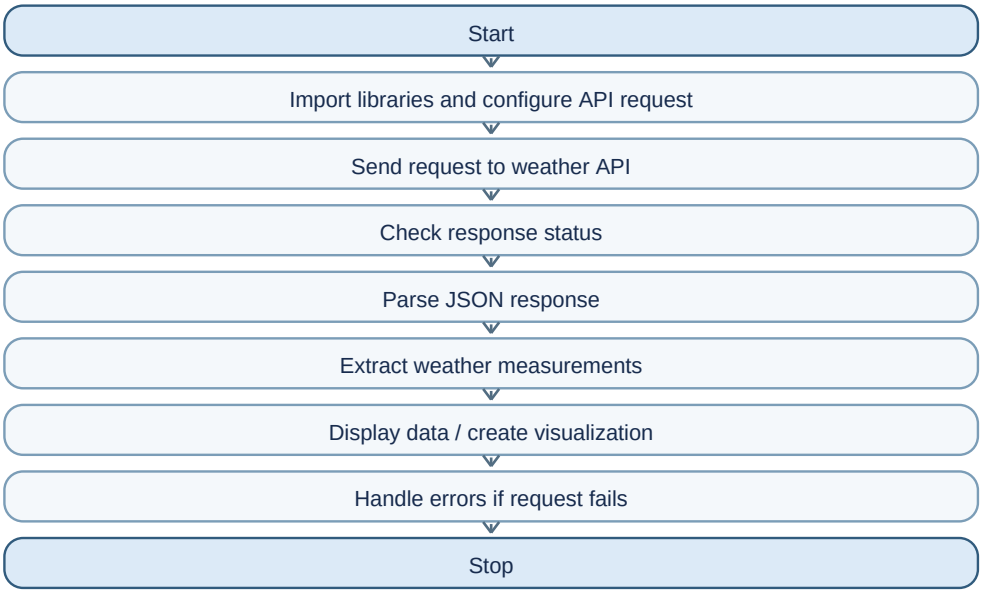
Problem statement

Fetch weather information from a weather API, parse the returned JSON data, and display useful weather details and visualizations.

Algorithm

1. Start the program.
2. Import requests, Pandas, and visualization libraries.
3. Set the city or location and configure the API request using a valid API key.
4. Send a request to the weather API.
5. Check the HTTP response status before processing the result.
6. Convert the JSON response into accessible Python data.
7. Extract relevant fields such as temperature, humidity, pressure, wind, or weather description.
8. Create a DataFrame if multiple observations are being analyzed.
9. Display the extracted values and create charts where appropriate.
10. Handle request errors or missing fields safely.
11. Stop.

Flowchart



Practical note: Keep API keys private. If a key was exposed publicly, revoke it and generate a replacement.

Practical 3: Data Cleaning and Preparation — Customer Churn

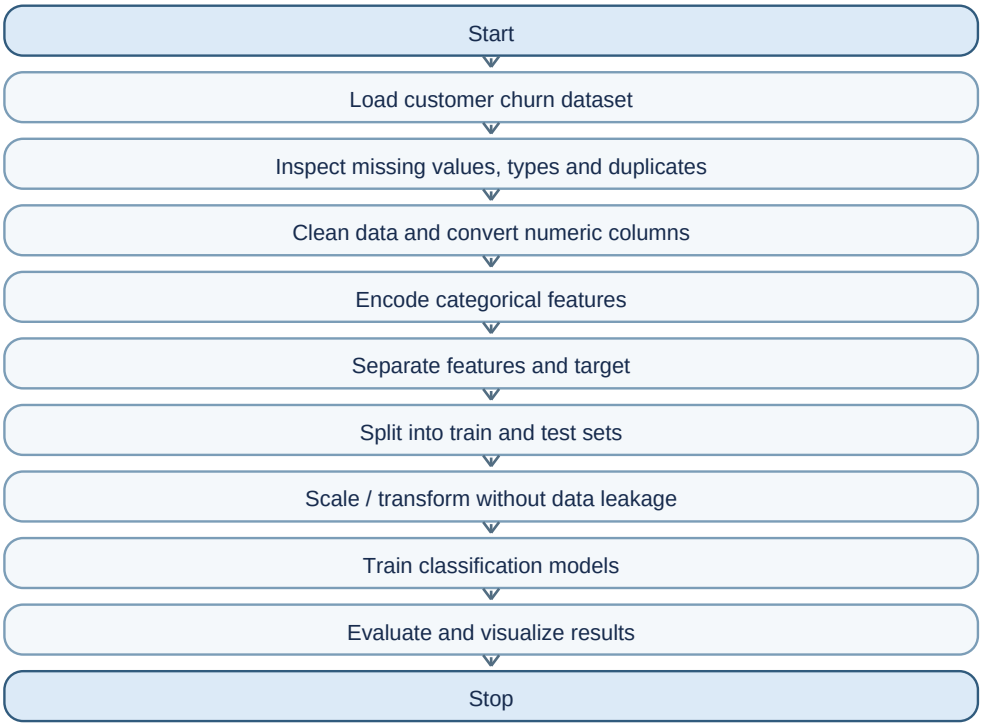
Problem statement

Prepare customer churn data by inspecting and cleaning records, encoding categorical variables, scaling numerical data where needed, and analyzing customer churn patterns. The discussed code also trains classification models.

Algorithm

1. Start the program.
2. Import Pandas, NumPy, plotting tools, and required scikit-learn classes.
3. Load the customer churn dataset.
4. Inspect dimensions, data types, missing values, and duplicate records.
5. Convert columns that should be numeric, handling invalid values.
6. Remove duplicates and handle missing values appropriately.
7. Drop identifier columns such as customerID when they do not provide useful predictive information.
8. Encode categorical variables into numerical representations.
9. Separate features and the churn target.
10. Split the data into training and testing sets before fitting preprocessing steps.
11. Fit scaling or other transformations on training data and apply them to test data.
12. Train classification models and evaluate results using suitable metrics and a confusion matrix.
13. Visualize important patterns and feature importance where appropriate.
14. Stop.

Flowchart



Practical note: Fit encoders and scalers correctly to avoid data leakage. The exact steps depend on the provided code and dataset.

Practical 4: Data Wrangling on Real Estate Market

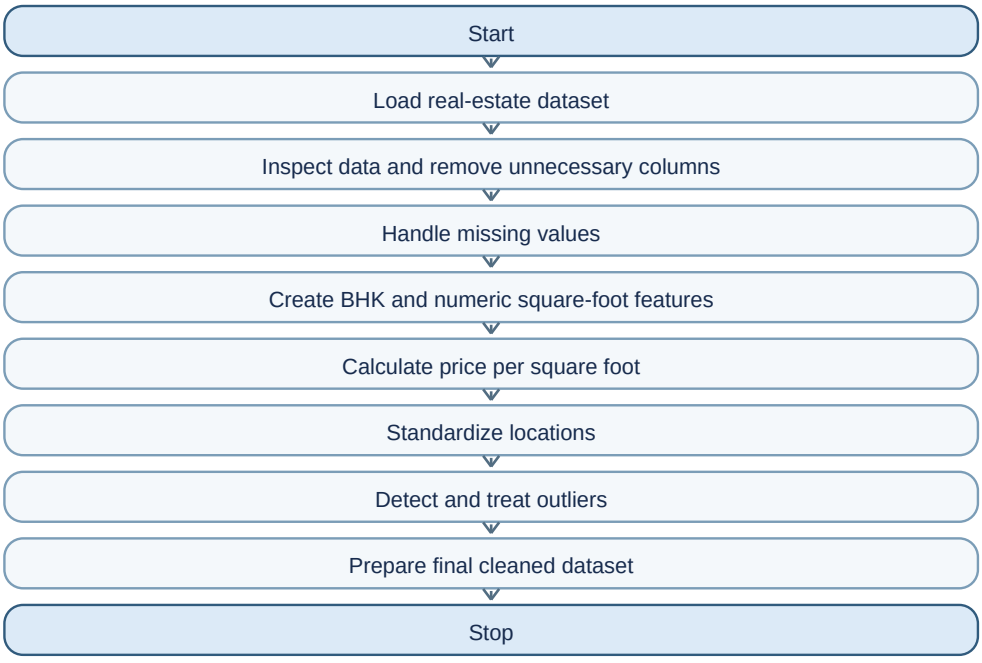
Problem statement

Clean and transform housing data, derive useful features such as BHK and price per square foot, treat outliers, and prepare the dataset for further analysis.

Algorithm

1. Start the program.
2. Import Pandas, NumPy, Matplotlib, and other required libraries.
3. Load the real-estate dataset and inspect its shape, columns, types, and sample rows.
4. Remove unnecessary columns such as area_type, society, balcony, or availability if specified.
5. Handle missing values and remove incomplete records where appropriate.
6. Extract the number of bedrooms (BHK) from the size column.
7. Convert total_sqft ranges into numeric values, for example by taking the average of the range.
8. Calculate price per square foot using price and total square footage, applying the correct unit conversion.
9. Standardize location names and group rare locations if required.
10. Remove implausible records, such as unusually low area per BHK.
11. Use plots and an interquartile-range method to inspect or filter outliers.
12. Prepare the cleaned feature and target data and split it if required.
13. Stop.

Flowchart



Practical note: Check the dataset's price units before applying a conversion such as multiplying by 100,000.

Practical 5: Data Visualization Using Matplotlib — Air Quality

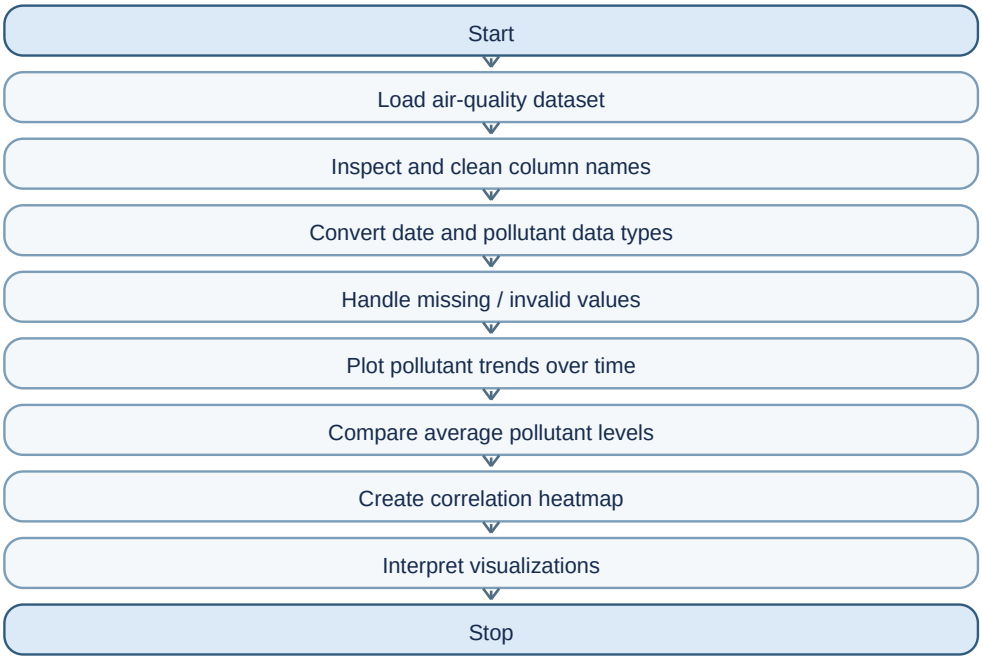
Problem statement

Explore air-pollution measurements and visualize pollutant levels, comparisons, and trends using graphs. The code discussed uses columns such as Date, CO(GT), NO2(GT), C6H6(GT), and NOx(GT) when available.

Algorithm

1. Start the program.
2. Import Pandas, NumPy, Matplotlib, Seaborn, and required preprocessing tools.
3. Load the air-quality CSV file using the correct separator and inspect column names.
4. Clean column names and remove empty columns.
5. Convert the date and pollutant columns to suitable data types.
6. Check missing values and handle them appropriately.
7. Plot pollutant measurements over time when a valid date column exists.
8. Calculate average pollutant levels and display them in a bar chart.
9. Calculate correlations between available pollutant columns.
10. Display a heatmap to visualize pollutant correlations.
11. Interpret the plots and stop.

Flowchart



Practical note: The dataset discussed had semicolon-separated columns and did not show an AQI column. Do not claim to plot AQI unless that column exists or AQI is calculated appropriately.

Practical 6: Data Aggregation — Retail Sales Performance

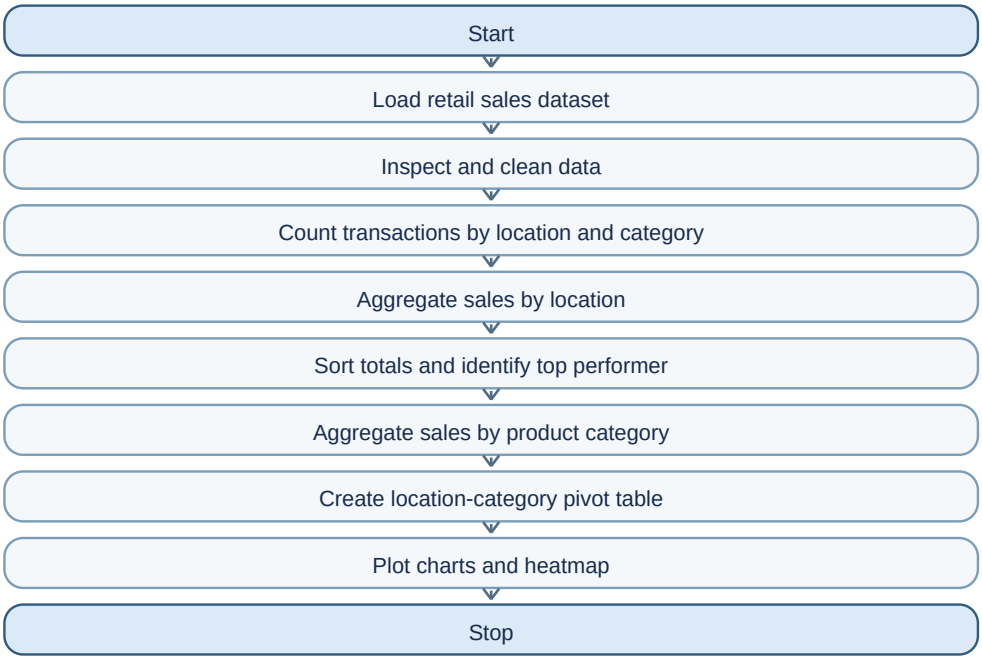
Problem statement

Aggregate retail transaction data to compare sales by shopping mall or region and product category, and identify the top-performing location.

Algorithm

1. Start the program.
2. Import Pandas, NumPy, Matplotlib, Seaborn, and other required libraries.
3. Load the retail sales dataset and inspect its structure.
4. Clean column names, remove duplicate rows, and handle missing or invalid sales amounts.
5. Count transactions by location and by product category.
6. Group records by location and sum the sales amount.
7. Sort location-wise totals in descending order to identify the top performer.
8. Group records by product category and calculate total category sales.
9. Create a pivot table of sales by location and category.
10. Visualize results using a pie chart, bar charts, grouped bar chart, and heatmap.
11. Interpret the sales performance and stop.

Flowchart



Practical note: If the dataset has shopping_mall rather than region, the analysis is mall-wise, not necessarily geographical-region-wise.

Practical 7: Time Series Analysis — Stock Market Data

Problem statement

Analyze historical stock prices to identify trends and patterns, forecast future closing prices, compare forecasting methods, and save the results.

Algorithm

1. Start the program.
2. Import Pandas, NumPy, Matplotlib, Seaborn, and time-series modeling libraries.
3. Load the stock-price dataset and inspect columns, data types, and missing values.
4. Convert Date to datetime format, sort chronologically, and set Date as the index.
5. Plot historical closing prices.
6. Calculate and plot moving averages such as MA-20, MA-50, and EMA-20.
7. Decompose the series into trend, seasonal, and residual components if appropriate.
8. If Volume is available, visualize its relationship with the closing price and calculate correlation.
9. Split observations chronologically into training and testing sets.
10. Select ARIMA parameters, fit the ARIMA model, and forecast the test period.
11. Fit Exponential Smoothing and forecast the same test period.
12. Compare forecasts with actual prices using RMSE and MAE.
13. Save actual and forecast values to a CSV file.
14. Stop.

Flowchart

Practical note: Forecasts are estimates, not guarantees. Choose decomposition settings and model parameters based on the data and validation results.